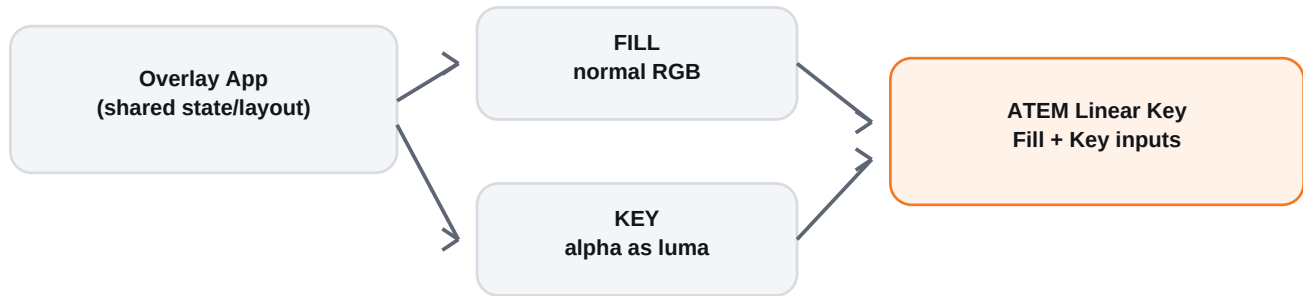


KEY + FILL OVERLAY

Implementation Specification for Codex

Purpose: build one broadcast overlay that exposes synchronized Fill and Key video variants for an ATEM linear key workflow.



One authoritative scene; both outputs share data, geometry, animation timing, masks and opacity.

Primary directive: The Key variant is an alpha mask rendered as grayscale. It is not a grayscale copy of the Fill image.

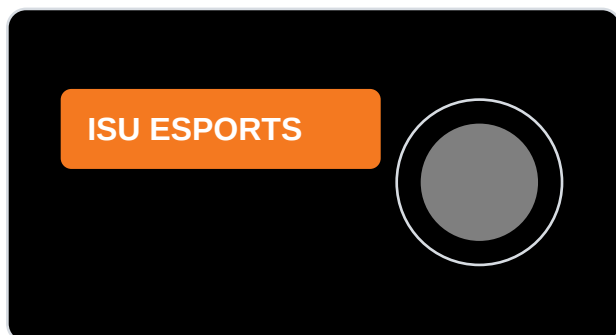
This document is written as a software implementation contract. Codex should treat the requirements and acceptance tests as authoritative unless the project already has a stronger existing architecture that preserves the same behavior.

Target	Recommended value
Canvas	1920 x 1080, fixed logical coordinate space
Frame rate	60 fps where the transport supports it
Fill transport	Normal RGB overlay composited over black
Key transport	Black background with scene alpha represented as 0-255 grayscale
ATEM mode	Linear key using separate Fill and Key inputs; Pre Multiplied Key ON for this recommended pipeline

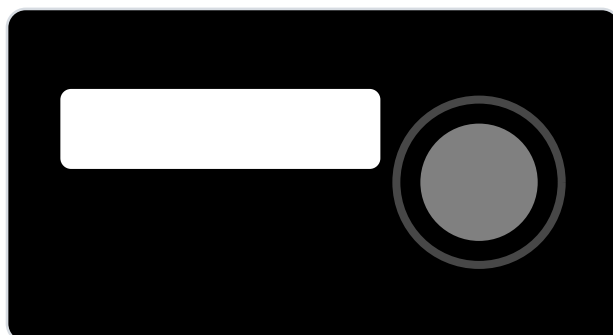
System model: browser overlay / graphics engine -> OBS or display outputs -> separate Fill and Key video inputs -> ATEM upstream or downstream linear key.

1. Signal Model Codex Must Implement

A linear key uses two synchronized video signals. The Fill signal supplies the color image. The Key signal supplies a grayscale mask that decides how much of the Fill is composited over the background.



FILL frame



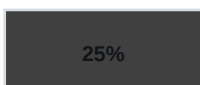
KEY frame

The Key signal represents alpha:

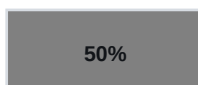
Expected KEY output for element opacity



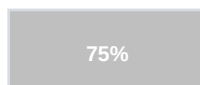
RGB approx 0



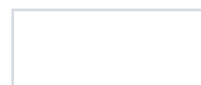
RGB approx 64



RGB approx 128



RGB approx 191



RGB approx 255

Black = transparent. White = fully opaque. Gray = partial transparency.

For normal source-over compositing, if an overlay pixel has alpha A from 0.0 to 1.0, the Key output for that pixel should be approximately $RGB = A \times 255$. The Key output should be neutral grayscale, so $R = G = B$.

Do not compute Key from Fill luminance. A dark orange logo can be fully opaque even though it is visually dark. Converting that logo to grayscale would incorrectly tell the ATEM that the logo is partly transparent.

The Fill and Key variants must be spatially identical. Every position, crop, scale, transform, opacity, mask, clip path, text layout, image asset, transition, animation phase and visibility decision must match.

2. Required Application Architecture

Build a single authoritative overlay scene and expose two rendering modes. Do not maintain separate Fill and Key layouts.

- **One state store:** scores, names, clocks, series state, logos, visibility flags and animations are shared.
- **One layout tree:** the same DOM/component tree or scene graph is used for both outputs.
- **Two output modes:** fill and key. A query parameter is sufficient, for example ?output=fill and ?output=key.
- **Deterministic animation timing:** animation start times come from shared state/timestamps, not each page load time.
- **Fixed logical resolution:** design at 1920 x 1080 and scale only at the final presentation layer if required.

Recommended route contract

```
GET /overlay?output=fill
GET /overlay?output=key
GET /overlay?output=test-fill
GET /overlay?output=test-key
```

At application start, validate the output parameter. Unknown values should fall back to Fill or display a clear diagnostics page. Do not silently render a malformed Key.

Recommended state shape

```
type OverlayState = {
  revision: number;
  updatedAtMs: number;
  animationStartMs?: number;
  visible: boolean;
  home: TeamState;
  away: TeamState;
  clock: ClockState;
  // ...other overlay data
};
```

Both variants subscribe to the same state feed. Any message that changes what is visible must reach both renderers with the same revision and timing metadata.

3. Rendering Algorithm

Recommended browser implementation: keep the overlay root transparent, place it over a black page, and apply a color-destroying filter only in Key mode. The filter should force every composited visible pixel to white while preserving alpha. When that result is composited over black, alpha becomes grayscale.

Reference CSS pattern

```
html, body {
  margin: 0;
  width: 100%;
  height: 100%;
  overflow: hidden;
  background: #000;
}

#overlay-root {
  position: relative;
  width: 1920px;
  height: 1080px;
  background: transparent;
}

html[data-output="key"] #overlay-root {
  filter: brightness(0) invert(1);
}
```

Why this works: the overlay root is first composited as normal transparent artwork. The filter removes RGB color but leaves alpha. White artwork over a black page then produces a luma value equal to the composited alpha.

Fill mode

- Render the actual overlay colors and artwork.
- Keep the page/output background black.
- Preserve intended opacity, masks, fades, antialiased edges, shadows and glows.
- Because transparent/semi-transparent pixels are displayed over black, the resulting Fill is shaped/premultiplied for the recommended ATEM configuration.

Key mode

- Use the exact same scene and animation state as Fill.
- Output pure black where scene alpha is 0.
- Output pure white where scene alpha is 1.
- Output proportional neutral gray where alpha is between 0 and 1.
- Do not preserve original scene color in the Key output.
- Do not threshold alpha unless a specific hard-key effect is requested.

Do not apply the key filter to the black page/background itself. Apply it only to the transparent overlay scene/root.

4. Handling Text, Images, SVG, Shadows and Effects

The Key variant must describe the final alpha footprint of the finished graphic. That includes soft edges and effects.

Asset / effect	Fill behavior	Key behavior
Text	Normal color and opacity	White glyphs; antialias pixels become gray
PNG/WebP with alpha	Normal image	Ignore RGB color; preserve source alpha silhouette
SVG	Normal fills/strokes	Final composited SVG becomes white while preserving alpha
50% opacity panel	Color at 50% over black	Approximately 50% gray
Drop shadow / glow	Normal effect	Effect contributes its alpha as gray falloff
Clip-path / mask	Normal clipped image	Identical clipped alpha footprint
Video with no alpha	Normal rectangle	White rectangle unless separately masked

Avoid or explicitly test unusual compositing features such as `mix-blend-mode`, `backdrop-filter`, browser effects that depend on content behind the overlay, and third-party canvas/WebGL scenes that clear to an opaque background. These can make the Key footprint differ from the Fill footprint.

If a canvas/WebGL component is used, it must support a transparent drawing buffer or expose a dedicated alpha-mask render pass. It must not render an opaque black rectangle inside `#overlay-root`, because Key mode would turn that rectangle white.

5. Synchronization and Broadcast Output Rules

Fill and Key are two views of one moving graphic. Even a small timing or scaling mismatch can create bright/dark fringes around moving edges.

- 1 **Same state revision:** Fill and Key should expose the current state revision in diagnostics. When debugging, verify both are on the same revision.
- 2 **Same animation epoch:** transitions should use a shared timestamp such as `animationStartMs`. Never start an animation from each page's local load time.
- 3 **Same viewport:** both outputs must use identical width, height, device scale assumptions and browser zoom.
- 4 **Same frame rate target:** configure both browser/capture outputs for the same frame rate.
- 5 **No independent random values:** particles, noise, random logo motion or generated IDs must share a seed/state or be disabled for broadcast key/fill output.
- 6 **No independent clocks:** countdowns and timers derive from shared timestamps, not separate `setInterval` counters.

OBS / output integration

- Create one Browser Source for Fill and one for Key, using the two output URLs.
- Set both to the same custom resolution and frame rate.
- Do not add color correction, scaling, crop, transform, sharpening or post-processing to only one of the two sources.
- If the outputs are sent to separate displays, SDI/HDMI paths or NDI paths, keep the complete transport chains equivalent wherever possible.

If transport latency differs between Fill and Key, the application cannot fully correct it. The hardware/software output paths must be matched or delayed to alignment downstream.

6. ATEM Configuration and Test Pattern

Blackmagic describes this as a **linear key** when the Fill and Key are separate sources. ATEM software exposes luma and linear key settings in a common luma-key menu.

ATEM setting	Recommended value
Key type	Luma / Linear key workflow
Fill Source	Video input carrying the Fill variant
Key Source	Video input carrying the Key variant
Pre Multiplied Key	ON for the recommended black-backed/premultiplied Fill pipeline
Invert Key	OFF when black = transparent and white = opaque

Codex should add a built-in test overlay

- Five opacity blocks: 0%, 25%, 50%, 75%, 100%.
- Large white and colored text with antialiased edges.
- A transparent PNG logo with curved edges.
- A soft shadow and a soft glow.
- A moving object that crosses the screen so Fill/Key timing can be checked.
- A diagnostic label showing output mode, state revision, logical resolution and animation timestamp.

The test Key output should visibly reproduce the alpha ramp shown earlier. If the 50% object is not approximately middle gray, the alpha pipeline is wrong.

7. Acceptance Tests

Codex should not consider the feature complete until all checks below pass.

#	Test	Pass condition
1	Transparent background	Key is pure black where no overlay pixels exist.
2	Opaque object	Key is pure white inside a fully opaque object.
3	50% object	Key is approximately RGB 128/128/128.
4	No color leakage	Every Key pixel is neutral grayscale: $R = G = B$.
5	Geometry parity	Fill and Key edges align pixel-for-pixel at the logical canvas size.
6	Soft edges	Text antialiasing, shadows and glows produce smooth gray ramps, not binary clipping.
7	Animation sync	Moving Fill and Key objects remain aligned through repeated transitions and a long-running test.
8	ATEM composite	No black/white fringe is visible over bright, dark and detailed backgrounds with premultiplied mode configured.
9	Resize stability	Both variants use the same fixed logical layout and scale identically.
10	Failure diagnostics	Invalid output mode or missing state feed is obvious in diagnostics; it does not silently output a plausible but wrong key.

Critical regression rule: a change to any overlay component must be visually checked in both Fill and Key modes. A component is not finished if it looks correct in Fill but produces the wrong alpha footprint.

8. Direct Instruction to Codex

The block below can be used as the implementation brief when Codex is working in the repository.

Implement synchronized broadcast Fill and Key variants for the overlay.

Requirements:

1. Keep one authoritative overlay scene, data store, component tree and animation state.
2. Add output modes `fill` and `key` selectable by URL/query parameter.
3. Fill mode renders the normal overlay over black.
4. Key mode renders the exact alpha footprint of the same scene as neutral grayscale: black = alpha 0, white = alpha 1, gray = partial alpha.
5. DO NOT make the Key by converting Fill colors to grayscale/luminance.
6. Prefer applying `filter: brightness(0) invert(1)` to the transparent, fully composited overlay root in Key mode, over a black page, so RGB is forced white while alpha is preserved. Do not include the black page background inside the filtered root.
7. Preserve alpha from text antialiasing, PNG/SVG assets, opacity, fades, masks, clip paths, shadows and glows.
8. Use shared timestamps/state revisions so Fill and Key animations remain aligned. No page-load-based animation timing or independent random values.
9. Add test-fill and test-key modes containing 0/25/50/75/100% alpha blocks, antialiased text, transparent imagery, shadow/glow and a moving object.
10. Add diagnostics showing output mode, state revision, logical resolution and animation epoch.
11. Target a fixed 1920x1080 logical canvas and keep both variants pixel-identical in geometry.
12. The expected switcher workflow is ATEM linear key with separate Fill and Key inputs and Pre Multiplied Key enabled for this black-backed Fill pipeline.

Do not duplicate overlay business logic between Fill and Key. Refactor shared rendering if necessary, then satisfy every acceptance test in the accompanying specification.

9. Technical References

Blackmagic Design - ATEM Television Studio Switchers Manual. The Keying section defines linear key as two sources: Fill plus a grayscale Key/Cut mask. It also explains premultiplied/shaped keys and notes that luma and linear key parameters share the luma-key menu.

https://documents.blackmagicdesign.com/UserManuals/ATEM_Television_Studio_Switchers_Manual.pdf

Blackmagic Design - ATEM Television Studio Features. Linear Key is described as separate Fill and Key sources, with the Key providing a grayscale mask.

<https://www.blackmagicdesign.com/products/atemtelevisionstudio/features>

Implementation note: the CSS alpha-mask method in this specification is a software design recommendation derived from standard alpha compositing behavior. It is not a Blackmagic-specific requirement. If the project renderer already exposes the final scene alpha directly, using that alpha as the Key is equally valid and can be more exact.